

A PROJECT REPORT ON

STUDY OF ML CLASSIFIERS FOR INTRUSION DETECTION SYSTEM

USING MACHINE LEARNING

Submitted in partial fulfilment of the requirements for the award of the degree of

DATA ANALYST INTERNSHIP

in

Appaxon Solutions

Submitted by

Vanshika Saini

Data Analyst Intern

Under the Guidance of

Vivek Jha

Internship Manager

[

CERTIFICATE

This is to certify that the project report entitled "Study of ML Classifiers for Intrusion Detection System Using Machine Learning" is a bona fide record of the work carried out by Vanshika Saini, in partial fulfilment of the requirements for the award of the certificate of Data analysis internship from Appaxon solutions.

The work presented in this report has not been submitted elsewhere for the award of any other degree or diploma, and to the best of my knowledge, represents the genuine effort of the student under my supervision.

Vivek Jha
Internship Manager
Appaxon Solutions

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to [Insert Guide Name Here], my project guide, for the constant support, valuable suggestions and patient guidance provided throughout the course of this project. The regular feedback and technical discussions were of great help in shaping the direction of this work.

I am also thankful to the faculty members of the [Insert Department Name Here] department for providing the necessary resources and a learning environment that made this project possible. I would like to extend my thanks to [Insert Head of Department Name Here], Head of Department, for the encouragement given during the course of this academic year.

Finally, I am grateful to my family and friends for their constant support and motivation, and to all those who directly or indirectly helped me in the completion of this project.

Vanshika Saini

ABSTRACT

Network intrusion detection is a core part of keeping computer networks secure, since attackers keep finding new ways to breach systems and steal or damage information. This project studies how six machine learning classifiers perform on the task of intrusion detection using the NSL-KDD dataset, a widely used benchmark for this problem. The work starts with an exploration of the training and testing data to understand the structure of network connection records, the split between normal and attack classes, and the presence of missing or duplicate records.

The categorical attributes `protocol_type`, `service` and `flag` were label-encoded, and the target class labels from both the training and testing sets were encoded jointly so that attack categories present only in the test set could also be represented. A constant-valued column, `num_outbound_cmds`, and an auxiliary column not needed for prediction were dropped. The remaining 35 features were standardised using a `StandardScaler` before being passed to distance-based and linear classifiers.

Six classifiers were trained and compared: Logistic Regression, K-Nearest Neighbors, Decision Tree, Random Forest, Support Vector Machine and Gaussian Naive Bayes. Each model was evaluated on training accuracy, testing accuracy, weighted precision, recall and F1-score, and the confusion matrix and classification report were examined for the best performing model. Random Forest gave the highest testing accuracy at 72.17 percent, followed closely by Logistic Regression and SVM, while Gaussian Naive Bayes performed far worse than the rest. Feature importance scores from the Random Forest model showed that `src_bytes`, `diff_srv_rate`, `same_srv_rate`, `flag` and `dst_host_serror_rate` were the most useful attributes for separating normal traffic from attacks.

These results line up with known characteristics of NSL-KDD, particularly the class imbalance between the majority classes and the many minority attack categories, and the fact that some attack types in the test set never appear during training. This report documents the full workflow, from data loading and cleaning through model comparison, and closes with a discussion of the limitations observed and possible directions for future work.

Keywords: Intrusion Detection System, Machine Learning, NSL-KDD Dataset, Classification, Random Forest, Support Vector Machine, Feature Importance, Network Security.

TABLE OF CONTENTS

1. Introduction	7
2. Problem Statement.....	7
3. Objectives	7
4. Scope of the Project.....	8
5. Literature Review	8
6. Methodology.....	8
6.1 Data Loading and Initial Checks	9
6.2 Data Quality Checks	9
6.3 Categorical Feature Encoding	9
6.4 Removing Redundant Columns	9
6.5 Feature Scaling.....	9
6.6 Model Training and Evaluation	9
7. Tools and Technologies.....	10
8. Dataset Description	10
8.1 Dataset Size	10
8.2 Feature Categories.....	10
8.3 Class Distribution and Imbalance	11
9. Exploratory Data Analysis	11
9.1 Class Distribution Plot.....	11
9.2 Structural Overview	12
9.3 Duplicate and Train-Test Consistency Checks	12
10. Feature Engineering.....	12
11. Model Building and Analysis	12
11.1 Logistic Regression.....	13
11.2 K-Nearest Neighbors (KNN)	13
11.3 Decision Tree Classifier	14
11.4 Random Forest Classifier	14
Confusion Matrix	15
Feature Importance	15
11.5 Support Vector Machine (SVM).....	16
11.6 Gaussian Naive Bayes	17
11.7 Consolidated Model Comparison	17
12. Results and Discussion	17
12.1 Overall Ranking	18
12.2 Why Test Accuracy Plateaus Around 70–72%	18
12.3 Random Forest Classification Report	19

13. Challenges Faced	19
Class Imbalance.....	19
Unseen Classes in the Test Set	19
Overfitting in Tree-Based Models.....	19
Duplicate Records and Fair Comparison.....	20
14. Conclusion.....	20
15. Future Scope	20
16. References	21

1. Introduction

Computer networks carry huge amounts of sensitive information, from personal communication to financial and organisational data. As more services move online, attackers keep finding new ways to disguise malicious traffic as normal activity. An Intrusion Detection System (IDS) is a security mechanism that monitors network traffic and flags connections or activities that look like an attack or unauthorised access attempt.

Traditional IDS tools work off a fixed set of signatures describing known attack patterns. This works for attacks that have already been documented, but fails against new or modified attacks whose signatures are not in the rule database. Machine learning based intrusion detection gets around this by learning to separate normal and malicious traffic from patterns in historical data instead of a fixed rule list, which lets a trained model generalise, at least to some extent, to attacks it has not seen exactly before.

This project studies how six commonly used machine learning classifiers perform on network intrusion detection using the NSL-KDD dataset, an improved and widely accepted version of the older KDD Cup 1999 dataset. The aim is not to propose a new algorithm but to build a complete pipeline, from loading the raw connection records through cleaning, encoding and training multiple classifiers, and to compare their performance using standard evaluation metrics so the practical strengths and weaknesses of each algorithm on this dataset become clear.

2. Problem Statement

Given a set of network connection records described by a mix of numeric and categorical attributes, such as connection duration, bytes transferred, protocol type and various traffic statistics, the problem is to classify each connection as either normal traffic or as one of several categories of network attack. This is made harder by the dataset being heavily imbalanced: a small number of classes such as normal and neptune make up the overwhelming majority of records, while many attack categories have only a handful of examples, and some attack types in the test set do not appear in the training set at all.

The core question this project tries to answer is practical: among a set of well-known, readily available classifiers, which one is best suited to this kind of multi-class, imbalanced network traffic classification problem, and what does the data tell us about why the classifiers perform the way they do.

3. Objectives

The objectives of this project, as reflected in the notebook developed for this study, are:

- To analyse the NSL-KDD network intrusion dataset and understand its structure, distribution and quality.
- To preprocess and clean the dataset, including handling categorical attributes, removing redundant columns, and scaling numerical features.
- To train multiple machine learning classifiers, namely Logistic Regression, K-Nearest Neighbors, Decision Tree, Random Forest, Support Vector Machine and Gaussian Naive Bayes, on the processed dataset.

- To compare the classification performance of these models using accuracy, precision, recall and F1-score.
- To identify the best performing model for this task and, through feature importance analysis, understand which network attributes contribute most to correct classification.

4. Scope of the Project

The scope of this project is limited to offline, batch-mode classification of network connection records using classical machine learning algorithms available in scikit-learn, on the standard NSL-KDD train/test split. It does not involve capturing or generating live network traffic, and deep learning architectures such as ANNs, CNNs or RNNs were not implemented, though they are noted later as a possible extension.

Class imbalance handling techniques such as SMOTE, ensemble stacking, and hyperparameter tuning through GridSearchCV were not applied in the current notebook; the classifiers were trained largely with default or lightly configured parameters so a fair, like-for-like comparison could be made between algorithms. The project also does not extend to deploying the trained model as a live service or dashboard.

5. Literature Review

Machine learning based intrusion detection research has largely been built around the KDD Cup 1999 dataset, created from network traffic captured in a simulated military network. Researchers later found this dataset had a very large number of duplicate records in both partitions, which biased classifiers towards frequently occurring records and produced misleadingly high accuracy figures. Tavallaee et al. proposed NSL-KDD as a refined version that removes duplicates and adjusts the difficulty distribution of records so that classifiers cannot simply memorise repeated patterns. NSL-KDD keeps the same 41 base features and attack categories as the original dataset, grouped into Denial of Service (DoS), Probe, Remote to Local (R2L) and User to Root (U2R), in addition to normal traffic.

A large body of prior work has applied classifiers such as Decision Trees, Naive Bayes, SVM, KNN and Random Forest to this dataset, generally reporting that tree-based ensembles perform competitively since they are less prone to overfitting than a single tree while still capturing non-linear relationships. Other studies have explored deep learning approaches such as autoencoders and RNNs, sometimes with better accuracy but at the cost of training complexity and interpretability. A recurring observation is that much of the difficulty of the NSL-KDD test set comes from attack types present only in the test partition, which is a deliberate design choice meant to test generalisation and which caps the achievable test accuracy of any classifier trained purely on the training partition. The results in this project, discussed in later sections, are consistent with this observation.

6. Methodology

The pipeline followed in this project is a standard supervised learning workflow adapted for a multi-class problem with categorical and numerical features. Each stage below is implemented as a distinct part of the notebook.

6.1 Data Loading and Initial Checks

The training and testing sets were loaded from `KDDTrain.parquet` and `KDDTest.parquet` using pandas. Parquet was likely chosen because it preserves column dtypes, including the category dtype used for `protocol_type`, `service` and `flag`, avoiding the need to redefine types after loading. The data was first inspected with `head()`, `info()`, `describe()` and `shape` to understand its structure before any cleaning was applied, and the training and testing sets were confirmed to share identical column names and dtypes.

6.2 Data Quality Checks

A null count with `isnull().sum()` showed no missing values in any of the 38 original columns. A duplicate check with `duplicated().sum()` found 5,843 duplicate rows out of 125,973 (4.64%), concentrated mostly in `ipsweep`, `nmap`, `normal`, `neptune` and `smurf`. These duplicates were analysed but not removed from the final notebook, which is useful context when interpreting the very high training accuracies obtained later.

6.3 Categorical Feature Encoding

The three categorical predictors, `protocol_type`, `service` and `flag`, were converted to numeric form with scikit-learn's `LabelEncoder`, fit on the training column and applied to both sets so the same mapping is used consistently. The target column, `class`, needed different treatment: the notebook confirmed 23 distinct classes in training against 38 in testing, with 17 attack categories appearing only in the test set. A single `LabelEncoder` was therefore fit on the union of class labels from both sets, then used to transform the class column in each set separately.

6.4 Removing Redundant Columns

`num_outbound_cmds` was found to hold the constant value zero across every record, so it carries no discriminative information and was dropped from both sets. An auxiliary column, `classnum`, not intended as a predictive input, was also dropped. This left 35 usable features across 125,973 training and 22,544 testing records.

6.5 Feature Scaling

A `StandardScaler` was fit on the training features and used to produce `X_train_scaled` and `X_test_scaled`, rescaling every feature to zero mean and unit variance. This matters for algorithms that are sensitive to feature scale, such as KNN, SVM and, to a lesser extent, Logistic Regression, all of which depend on distance calculations or gradient-based optimisation that gets distorted when features vary widely in magnitude. Decision Tree and Random Forest were trained on the unscaled features instead, since tree splits compare thresholds within a single feature and are unaffected by overall scale.

6.6 Model Training and Evaluation

Six classifiers, Logistic Regression, KNN, Decision Tree, Random Forest, SVM and Gaussian Naive Bayes, were trained on the appropriately scaled or unscaled version of the data and evaluated on both training and testing sets for accuracy. Random Forest was examined further with a classification report, confusion matrix and feature importance ranking. A consolidated comparison across all six models was then produced using accuracy, weighted precision, weighted recall and weighted F1-score, computed through a small `evaluate_model` helper function defined in the notebook.

7. Tools and Technologies

The project was built as a Jupyter notebook on Python 3. The table below lists the main libraries used.

Category	Tool / Library	Purpose
Programming Language	Python 3	Core language for the project
Environment	Google Colab (Jupyter Notebook)	Interactive notebook for writing and running code
Data Handling	pandas	Loading, cleaning and manipulating tabular data
Numerical Computing	NumPy	Array operations and numerical utilities
Visualisation	Matplotlib, Seaborn	Bar plots, count plots and confusion matrix heatmap
Machine Learning	scikit-learn	Preprocessing (LabelEncoder, StandardScaler), classifiers, evaluation metrics
File Format	Apache Parquet	Storage format for the training and testing datasets

Table 7.1: Tools and technologies used in the project

The notebook also imports LightGBM, XGBoost, RFE and tabulate, but none of these were actually used to train or evaluate a model; only the six classifiers above were trained. These imports are best read as preparation for the extensions discussed in Section 15, Future Scope.

8. Dataset Description

The dataset used is NSL-KDD, supplied as KDDTrain.parquet and KDDTest.parquet. It is a refined version of the older KDD Cup 1999 dataset, created to remove redundant records and give a more balanced basis for evaluating intrusion detection algorithms. Each record represents a single network connection described by basic connection attributes (duration, protocol, bytes transferred), content-based attributes (failed logins, root shell access) and traffic statistics computed over recent connections (error rates, same-service rates).

8.1 Dataset Size

Partition	File	Records	Columns
Training set	KDDTrain.parquet	125,973	38
Testing set	KDDTest.parquet	22,544	38

Table 8.1: Size of the training and testing partitions

8.2 Feature Categories

After the two non-informative columns were removed (Section 6.4), 35 attributes remained for training. These fall broadly into four groups:

- Basic connection features: duration, protocol_type, service, flag, src_bytes, dst_bytes, land, wrong_fragment, urgent.
- Content features: hot, num_failed_logins, num_compromised, root_shell, su_attempted, num_root, num_file_creations, num_shells, num_access_files, is_host_login, is_guest_login.

- Time-based traffic features (2-second window): count, serror_rate, srv_serror_rate, srv_error_rate, same_srv_rate, diff_srv_rate.
- Host-based traffic features (last 100 connections to the same host): dst_host_count, dst_host_same_srv_rate, dst_host_diff_srv_rate, dst_host_same_src_port_rate, dst_host_srv_diff_host_rate, dst_host_serror_rate, dst_host_srv_serror_rate, dst_host_error_rate, dst_host_srv_error_rate.

The target column, class, has 23 distinct labels in training and 38 in testing, meaning the test set introduces attack categories the model never sees during training. This point comes up again in the results discussion, since it directly caps achievable test accuracy.

8.3 Class Distribution and Imbalance

The class distribution in the training set is heavily skewed. normal accounts for 67,343 records and neptune for a further 41,214, together close to 86% of the training set. The remaining classes range from a few thousand records (satan 3,633; ipsweep 3,599; portsweep 2,931) down to single digits (spy 2; perl 3; phf 4). Several categories such as apache2, httptunnel, mscan and mailbomb have zero occurrences in training, since they belong to the set of attacks seen only in testing.

Class	Record Count	Approx. Share
normal	67,343	53.5%
neptune	41,214	32.7%
satan	3,633	2.9%
ipsweep	3,599	2.9%
portsweep	2,931	2.3%
smurf	2,646	2.1%
nmap	1,493	1.2%
back	956	0.8%
teardrop	892	0.7%
warezclient	890	0.7%

Table 8.2: Ten most frequent classes in the training set

The remaining thirteen training classes each contribute well under 1% of records, several with fewer than five examples in total. This long-tail distribution is the single biggest factor shaping model performance, since every classifier tested will naturally learn the dominant classes well while struggling with rare ones purely from lack of exposure.

9. Exploratory Data Analysis

EDA was carried out to understand the shape and quality of the data before modelling, through descriptive statistics and the visualisation below.

9.1 Class Distribution Plot

Purpose: Visualise how connection records are spread across the 23 training classes.

Observation: The normal and neptune bars dominate the plot; bars for most other attack categories are barely visible on the same scale.

Insight: The dataset is highly imbalanced, and any classifier trained on it will be biased toward recognising the majority classes.

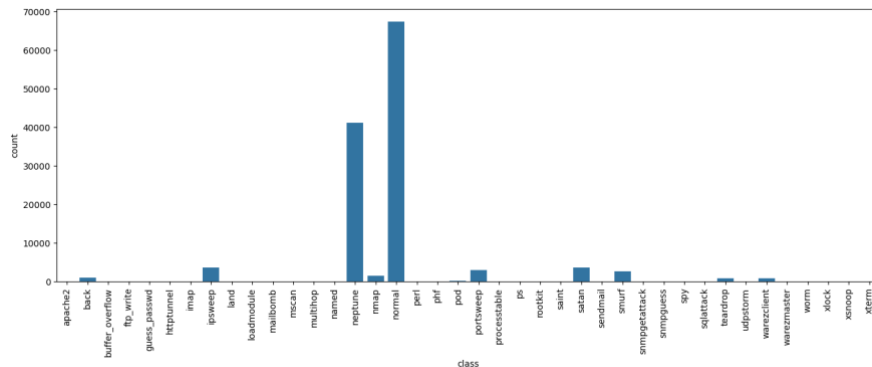


Figure 9.1: Distribution of connection classes in the training set

9.2 Structural Overview

info() confirmed the training set is a mix of integer, unsigned-integer and category dtypes, with protocol_type, service and flag as categories and the rest numeric. describe() showed src_bytes and dst_bytes have extremely wide ranges, reaching into the hundreds of millions despite a median close to zero for both, a heavily right-skewed distribution typical of network byte counts where most connections transfer very little data and a few transfer a great deal.

9.3 Duplicate and Train-Test Consistency Checks

The 4.64% duplicate rows identified earlier are concentrated in ipsweep, nmap, normal, neptune, smurf, warezclient, satan, pod and land, not spread evenly across classes. train.columns.equals(test.columns) and train.dtypes.equals(test.dtypes) both returned True, confirming the two files are structurally consistent before any transformation was applied to them.

10. Feature Engineering

No new derived features were constructed; all 41 base NSL-KDD features were already engineered by the dataset's creators from raw packet captures. What this project does is better described as feature preparation: label encoding of the categorical attributes and target class, and feature scaling with StandardScaler. Dropping the constant column num_outbound_cmds and the auxiliary classnum column is feature selection rather than construction. Feature importance from the trained Random Forest (Section 11.4) was used diagnostically to see which of the 35 features mattered most, but it was not fed back into the pipeline to select a reduced feature set. Recursive feature elimination, imported via scikit-learn's RFE class, was not applied to any of the six models and is listed under future work.

11. Model Building and Analysis

Six classifiers were trained on the preprocessed data: a linear model, a distance-based model, two tree-based models, a margin-based model and a probabilistic model. Each subsection below gives a

short introduction, why it was chosen, how it performed, and its advantages and limitations on this dataset.

11.1 Logistic Regression

Logistic Regression is a linear classification algorithm and was used here as a baseline against which the more complex classifiers could be compared.

Why selected: As a cheap linear model, it gives a reference point for judging whether the non-linear models actually add value over a linear decision boundary.

Implementation: `LogisticRegression(max_iter=1000, random_state=42)`, trained on `X_train_scaled`. `max_iter` was raised from the default so the optimiser had enough iterations to converge.

Performance: Training accuracy 97.74%, testing accuracy 71.78%. The relatively small train-test gap, compared to the tree-based models, suggests the linear model overfits less, even though its raw test accuracy trails the best model only slightly.

Advantages:

- Fast to train and easy to interpret through per-feature weights.
- Less prone to overfitting than more flexible models, particularly on the majority classes.

Limitations:

- Assumes an approximately linear relationship between features and log-odds, which may not capture more complex traffic patterns.
- Like every model here, performance on rare attack classes is limited by how few training examples those classes have.

11.2 K-Nearest Neighbors (KNN)

KNN is an instance-based algorithm that classifies a record by looking at the classes of its nearest neighbours in feature space rather than learning explicit parameters.

Why selected: It represents a fundamentally different approach from the parametric models in this study and is a natural fit here, since connections of the same attack type often share similar traffic statistics.

Implementation: `KNeighborsClassifier(n_neighbors=5)`, trained on `X_train_scaled` since KNN relies directly on distance calculations that need standardised inputs.

Performance: Training accuracy 99.69%, testing accuracy 70.83%. The very high training accuracy is expected with $k=5$, since a training point is almost always classified correctly by its own near-identical neighbours; the drop on the test set reflects weaker generalisation to genuinely new patterns.

Advantages:

- Simple to understand, with no explicit training phase beyond storing the data.
- Can capture non-linear class boundaries that a linear model cannot.

Limitations:

- Prediction is slow on large datasets, since every prediction compares against many stored records.

- Sensitive to class imbalance, since majority classes dominate the neighbourhood of most points.

11.3 Decision Tree Classifier

A Decision Tree builds a set of if-then rules, splitting the data repeatedly on the feature and threshold that best separates the classes at each step.

Why selected: Decision Trees are interpretable, cheap to compute, and need no feature scaling. They were also included as a stepping stone to Random Forest, since a forest is essentially an ensemble of trees, and comparing the two shows what the ensemble adds.

Implementation: `DecisionTreeClassifier(criterion='entropy', random_state=42)`, trained on the unscaled `X_train`.

Performance: Training accuracy 99.99%, essentially a perfect fit, but testing accuracy only 70.34%, the lowest of the six models. With no maximum depth set, the tree grew until it could separate almost every training record, including duplicates and noise, at the cost of generalising poorly.

Advantages:

- Easy to interpret, since each prediction traces back through a sequence of simple decisions.
- Needs no feature scaling and handles mixed feature types naturally.

Limitations:

- Highly prone to overfitting when grown without depth constraints, as seen clearly here.
- A single tree is unstable; small changes in training data can produce a noticeably different tree.

11.4 Random Forest Classifier

Random Forest is an ensemble method that builds many decision trees on different random subsets of data and features, and combines their predictions by majority vote.

Why selected: Given the overfitting expected of a single tree, Random Forest was chosen to reduce that overfitting through averaging, and is a commonly recommended baseline for tabular classification.

Implementation: `RandomForestClassifier(n_estimators=100, random_state=42)`, trained on unscaled features. `feature_importances_` was used afterward to rank the 35 input features.

Performance: Training accuracy 99.99%, testing accuracy 72.17%, the highest of all six classifiers. Despite training accuracy nearly identical to the single Decision Tree, testing accuracy is noticeably higher, consistent with the averaging effect across 100 trees reducing variance.

Advantages:

- More robust to overfitting than a single tree, reflected in its higher test accuracy here despite similar training accuracy.
- Provides a built-in feature importance measure that adds model-level interpretability.

Limitations:

- More expensive to train and query than a single tree, since it builds and runs many trees.

- Still shows a real gap between training and testing accuracy, so some overfitting remains despite ensembling.

Confusion Matrix

Purpose: Show where the Random Forest model's predictions land relative to the true class labels on the test set.

Observation: Most predictions fall along the diagonal for the dominant classes, but there is considerable confusion among the many smaller attack categories.

Insight: The model performs strongly on well-represented classes but struggles wherever training support is thin, consistent with the class imbalance discussed in Section 8.3.

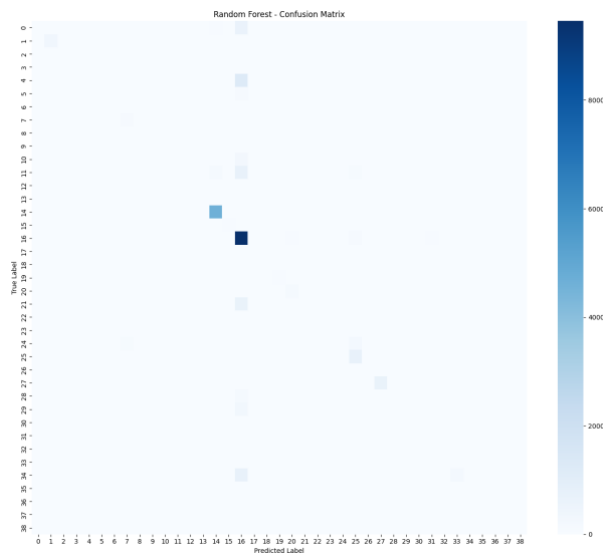


Figure 11.1: Confusion matrix for the Random Forest classifier on the test set

Feature Importance

Purpose: Rank the 35 input features by how much they contributed to the Random Forest's split decisions.

Rank	Feature	Importance
1	src_bytes	0.1189
2	diff_srv_rate	0.1031
3	same_srv_rate	0.0944
4	flag	0.0804
5	dst_host_serror_rate	0.0690
6	serror_rate	0.0634
7	dst_bytes	0.0576
8	dst_host_diff_srv_rate	0.0444
9	count	0.0437
10	protocol_type	0.0395

Table 11.1: Top 10 features by Random Forest importance score

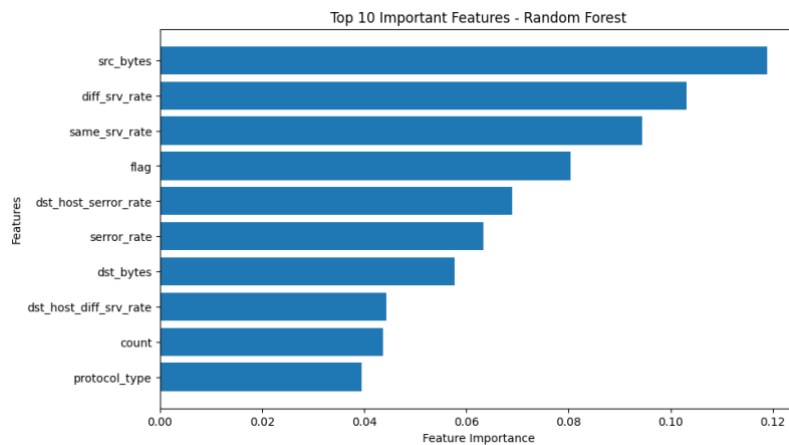


Figure 11.2: Top 10 important features identified by the Random Forest classifier

Observation: src_bytes and dst_bytes rank highest, followed by the service and error-rate statistics computed over recent connections.

Insight: Traffic volume is highly discriminative, which fits attacks like DoS floods that generate unusual byte counts; the strong ranking of the window-based rate features shows the engineered statistical attributes in NSL-KDD carry much of the useful signal.

11.5 Support Vector Machine (SVM)

SVM is a margin-based classifier that looks for the decision boundary separating classes with the largest possible margin.

Why selected: It is a well-established classifier that often does well on moderate-sized datasets with many features, and gives a useful comparison against both the linear model and the tree-based models since it can capture non-linear boundaries through a kernel.

Implementation: SVC(kernel='rbf', random_state=42), trained on X_train_scaled. The rbf kernel handles the multi-class problem internally via a one-versus-one strategy.

Performance: Training accuracy 99.03%, testing accuracy 71.07%, third among the six models, just behind Logistic Regression and Random Forest.

Advantages:

- Models non-linear decision boundaries through the kernel choice, without hand-engineering non-linear features.
- Tends to generalise well when class margins are reasonably clear, reflected in its competitive test accuracy here.

Limitations:

- Training time grows considerably with the number of records, a real concern given this dataset's size.
- Performance and training time are both sensitive to kernel and parameter choice, none of which were tuned here.

11.6 Gaussian Naive Bayes

Gaussian Naive Bayes is a probabilistic classifier based on Bayes' theorem that assumes features are conditionally independent given the class, and that each numeric feature is normally distributed within a class.

Why selected: It was included as a lightweight, fast baseline, and to see how a model built on a strong independence assumption performs on a dataset where several features, such as the error-rate and same-service-rate columns, are known to be correlated rather than independent.

Implementation: `GaussianNB()` with default parameters, trained on `X_train_scaled`.

Performance: Training accuracy only 45.86%, testing accuracy 31.26%, by far the weakest of the six classifiers. This fits the strong violation of the independence assumption in this dataset: several traffic-based features are derived from the same underlying connection statistics and are closely related to each other.

Advantages:

- Extremely fast to train and predict, needing only simple per-class statistics for each feature.
- Needs very little training data to estimate its parameters compared to more complex models.

Limitations:

- Performance suffers considerably when the independence assumption is strongly violated, as seen here.
- By far the weakest results in this comparison; not a suitable choice for this dataset as it stands.

11.7 Consolidated Model Comparison

Model	Training Accuracy	Testing Accuracy
Logistic Regression	0.9774	0.7178
K-Nearest Neighbors	0.9969	0.7083
Decision Tree	0.9999	0.7034
Random Forest	0.9999	0.7217
Support Vector Machine	0.9903	0.7107
Gaussian Naive Bayes	0.4586	0.3126

Table 11.2: Training and testing accuracy of all six classifiers

Apart from Gaussian Naive Bayes, every model reaches over 97% training accuracy, yet testing accuracy clusters much more tightly, between roughly 70% and 72%. High, closely bunched training accuracy next to a lower, similarly bunched testing accuracy is a strong sign that the ceiling on test performance here is set by the data itself, not by the choice of algorithm — the point examined next in Results.

12. Results and Discussion

Beyond plain accuracy, each classifier was evaluated using weighted precision, recall and F1-score on the test set, computed through a helper function in the notebook. Weighted averaging combines per-class metrics in proportion to class support, which suits an imbalanced dataset like this one since it stops the many tiny classes from being weighted equally with the two dominant ones.

Model	Accuracy	Precision	Recall	F1-Score
Random Forest	0.7217	0.5964	0.7217	0.6218
Logistic Regression	0.7178	0.5932	0.7178	0.6175
SVM	0.7107	0.6096	0.7107	0.6168
KNN	0.7083	0.6040	0.7083	0.6147
Decision Tree	0.7034	0.5498	0.7034	0.6075
Gaussian Naive Bayes	0.3126	0.3886	0.3126	0.2842

Table 12.1: Weighted precision, recall and F1-score, sorted by accuracy

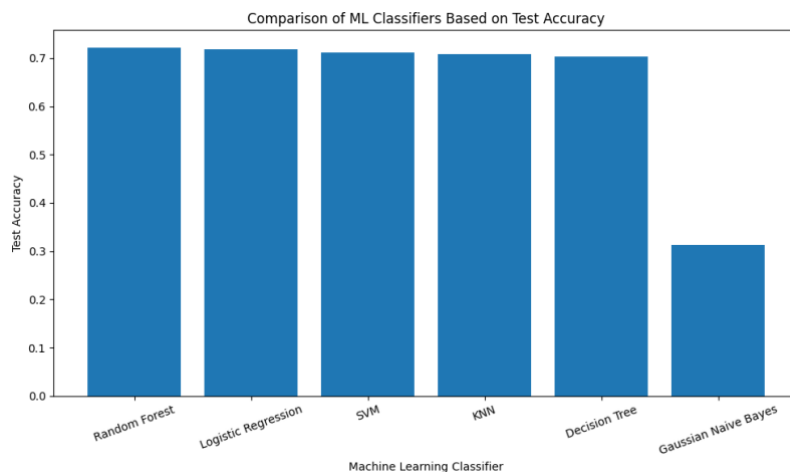


Figure 12.1: Comparison of test accuracy across all six classifiers

12.1 Overall Ranking

Random Forest comes out on top by every metric in Table 12.1, with 72.17% testing accuracy and the best F1-score. Logistic Regression and SVM follow within about a percentage point, while KNN and Decision Tree trail slightly further. Gaussian Naive Bayes stands well apart, at less than half the accuracy of the other five, consistent with the independence-assumption issue discussed in Section 11.6. Although SVM and KNN edge out Random Forest on weighted precision alone, Random Forest still gives the best balance of precision and recall, reflected in its leading F1-score, which is the more informative single number on an imbalanced multi-class problem like this one.

12.2 Why Test Accuracy Plateaus Around 70–72%

Five of the six models land within a two-point band on testing accuracy despite training accuracies ranging from about 97% to nearly 100%, which points to a ceiling set by the data rather than by algorithm choice. Two properties of the NSL-KDD test set explain most of this. First, seventeen attack categories appear only in the test set (Section 6.3); no classifier can be expected to recognise a class it has never seen an example of, and the best it can do is misclassify such a record as the known class

it most resembles. Since these unseen categories make up a real share of the 22,544 test records, they cap the maximum achievable accuracy regardless of the algorithm used. Second, the class imbalance documented in Section 8.3 means several rare attack types have only a handful of training examples, which is rarely enough for any classifier to learn a reliable boundary for them — visible directly in the Random Forest classification report below, where several low-support classes show zero precision and recall.

12.3 Random Forest Classification Report

A small number of well-represented classes, such as normal and neptune, are classified with precision and recall above 0.90, while a considerable number of the remaining 38 class labels show precision and recall of exactly zero.

Metric	Value
Overall accuracy	0.72 (22,544 test samples)
Macro average precision	0.29
Macro average recall	0.28
Macro average F1-score	0.25
Weighted average precision	0.60
Weighted average recall	0.72
Weighted average F1-score	0.62

Table 12.2: Summary of the Random Forest classification report on the test set

The gap between macro-averaged scores (every class weighted equally) and weighted-averaged scores (dominated by the large classes) is itself a clear illustration of the imbalance problem: a macro F1 of only 0.25 next to a weighted F1 of 0.62 shows the model's decent overall accuracy comes mainly from a handful of large classes, while it performs poorly across most of the 38 individual labels, most of which have very few test examples.

13. Challenges Faced

Class Imbalance

The heavy skew toward normal and neptune made it hard for every classifier to learn the rarer attack categories well. This is a data-level issue rather than a modelling mistake, and it was not fully addressed within this project's scope since techniques such as SMOTE or class-weighted loss were not applied to the final models.

Unseen Classes in the Test Set

Finding that the test set contains 17 attack categories absent from training required fitting the class encoder on the union of both sets rather than on training labels alone, since a LabelEncoder fit only on training would fail on unseen test labels. The underlying limitation, that no classifier can correctly predict a class it never saw during training, remains built into the results.

Overfitting in Tree-Based Models

Both Decision Tree and, to a lesser degree, Random Forest showed training accuracy near 100% alongside testing accuracy roughly 27 to 30 points lower. Recognising this as overfitting, rather than mistaking high training accuracy for strong model performance, was an important part of interpreting the results correctly.

Duplicate Records and Fair Comparison

The 4.64% duplicate rows, concentrated in a handful of classes, add a further source of potential bias since a model can appear to fit these repeated patterns well without that reflecting real generalisation. Because some algorithms need scaled input (Logistic Regression, KNN, SVM, Naive Bayes) while the tree-based models do not, care was also needed to apply StandardScaler only where appropriate and use the same encoded feature set across all six models so the comparison in Section 12 stayed fair.

14. Conclusion

This project studied how six standard machine learning classifiers perform on network intrusion detection using the NSL-KDD dataset. The data was found to have no missing values but a meaningful share of duplicate records and substantial class imbalance, with normal and neptune together making up most of the training set. After encoding the categorical attributes and class label, dropping a constant-valued column, and standardising features where appropriate, Random Forest came out as the best of the six models, with 72.17% testing accuracy and the best weighted F1-score. Logistic Regression and SVM followed closely, Decision Tree showed clear overfitting, and Gaussian Naive Bayes performed considerably worse than the rest, most likely due to strong correlation among several traffic-based features violating its independence assumption.

Feature importance from Random Forest highlighted src_bytes, diff_srv_rate, same_srv_rate, flag and dst_host_serror_rate as the most useful attributes for telling normal and malicious traffic apart, which fits the intuitive idea that traffic volume and service-usage patterns are strong attack indicators. At the same time, the consistently limited testing accuracy across nearly every classifier, despite very high training accuracy, points to class imbalance and the presence of previously unseen attack types in the test set as the main factors limiting performance, rather than any specific weakness of the algorithms themselves. Overall, the results show ensemble methods like Random Forest offer a modest but real edge over a single tree or a purely linear model on this kind of imbalanced, multi-class problem, and confirm why NSL-KDD remains a genuinely challenging benchmark even for well-established classical algorithms.

15. Future Scope

- Hyperparameter tuning: use GridSearchCV or RandomizedSearchCV to search for better parameter settings for each classifier instead of relying on defaults.
- Advanced ensembles: try gradient boosting methods such as XGBoost, LightGBM and CatBoost, already imported but unused in the notebook, which often outperform a plain Random Forest on tabular data.
- Class imbalance handling: apply oversampling techniques such as SMOTE, or class-weighted loss functions, to improve detection of the minority attack classes.

- Feature selection: run a more systematic feature selection process, for instance with the RFE technique already imported into the notebook, to check whether a smaller feature set can match current accuracy with lower complexity.
- Cross-validation and deployment: use k-fold cross-validation for a more reliable performance estimate, and explore packaging the trained model behind a simple monitoring dashboard for interactive use.

16. References

- [1] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in Proc. 2nd IEEE Symp. Computational Intelligence for Security and Defense Applications (CISDA), Ottawa, ON, Canada, 2009, pp. 1-6.
- [2] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.
- [3] W. McKinney, "Data structures for statistical computing in Python," in Proc. 9th Python in Science Conf. (SciPy), Austin, TX, USA, 2010, pp. 56-61.
- [4] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5-32, Oct. 2001.
- [5] C. Cortes and V. Vapnik, "Support-vector networks," Machine Learning, vol. 20, no. 3, pp. 273-297, Sep. 1995.
- [6] J. D. Hunter, "Matplotlib: A 2D graphics environment," Computing in Science & Engineering, vol. 9, no. 3, pp. 90-95, 2007.
- [7] M. L. Waskom, "seaborn: statistical data visualization," Journal of Open Source Software, vol. 6, no. 60, p. 3021, 2021.
- [8] NSL-KDD Dataset, Canadian Institute for Cybersecurity, University of New Brunswick. [Online]. Available: <https://www.unb.ca/cic/datasets/nsl.html>